2026 Shenzhen International FinTech Competition

International Track | Topic F Agent Wallet

mandate.

Technical Report

An AI agent for approved business bill payments

ChainSentinel Bangladesh

TEAM MEMBERS

MD ARIF SHEKH — Team Leader
Computer Science and Engineering | Undergraduate Year 4
International University of Business Agriculture and Technology (IUBAT)

MD MEHERAB HOSSAIN TALUKDER — Team Member
Computer Science and Engineering | Undergraduate Year 4
International University of Business Agriculture and Technology (IUBAT)

IRIN KHATUN — Team Member
Accounting & Information Systems | Undergraduate Year 4
Jagannath University

20 October 2026

Local chain and Ethereum Sepolia | Synthetic bills and MDT test tokens

01 Problem and Product

Mandate helps a business owner turn reviewed supplier bills into a controlled payment batch. The owner checks the source and approves precise payment details. An AI assistant interprets the request; application code selects eligible bills; a restricted executor sends them through a vault contract. This report explains the implementation and how to run it.

The business task

A small agency may have several invoices ready while needing to retain funds for other expenses. Paying them one by one requires repeated wallet actions. Delegating a broad wallet creates a different problem: an assistant might misunderstand a message, choose the wrong recipient or process the same bill twice. Mandate gives the assistant a limited bill-payment task with visible owner control.

The owner workflow

The owner imports a CSV or text-based PDF, compares its source with the draft fields, checks the trusted vendor and signs a batch approval. A later conversation can ask about the bills, resolve an unclear request or request a payment plan. The interface shows the proposed batch before execution. After processing, it displays confirmed receipts and a reason for each deferred bill.

What the project contributes

The useful part is the connected workflow: source review, conversational assistance, deterministic planning and receipt-based reporting. The AI helps the owner express the task; it does not replace the code that calculates eligibility. Bulk approval and AI assistance solve different parts of the process, so we do not attribute every reduction in wallet actions to the model.

Orbit Studio is the fictional business used to exercise the flow. Its supplier identities, invoices and token are controlled demonstration materials. The prototype supports one business and one vault. It does not implement trading, bridges, lending or general wallet automation.

How the three reports fit together

The Security Self-Assessment is the detailed account of permissions, threat tests, privacy and residual risks. Execution Evidence is the record of measured results, observed failures and public transaction receipts. Those reports are referenced here rather than reproducing their tables.

Implementation sources: app/public/, app/client.mjs, app/service.mjs, app/agent.mjs and contracts/MandateVault.sol. Product requirements: REQUIREMENTS.md (R01–R15).

02 System Architecture

The browser calls a Node.js service that manages versioned drafts, model proposals, plans and transaction records in SQLite. In the hosted configuration, a Render gateway fronts the loopback service and saves workspace snapshots to Neon. The same application can run locally. The security assessment explains the hosted authentication and storage protections.

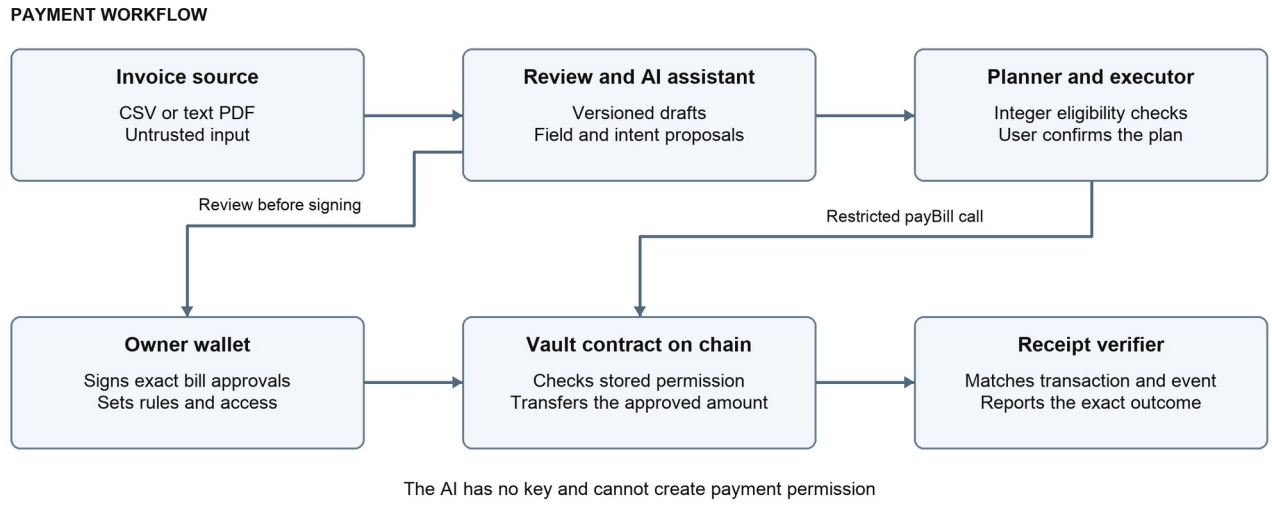


Figure 1 Invoice information enters the review flow; owner approvals and contract checks define the payment path.

Component	Implementation responsibility
Browser workspace	Upload/source review, exact action preview, wallet requests, conversation and activity views.
Service and SQLite store	Version draft changes, bind plans to current records and retain original transaction references.
Model adapter	Send bounded extraction or interpretation input; validate the returned structured proposal.
Planner and executor	Order eligible bills, build a fixed payment queue and reconcile outcomes.
Vault and verifier	Read approved payment data, enforce transaction conditions and match receipt events.

The system separates document state from chain state. A draft stores the current interpretation of an invoice; an approval is an on-chain record. Editing a draft therefore requires a new review before it can become a new executable version. The UI shows those states separately.

Source map: app/store.mjs and app/service.mjs; app/model.mjs and app/agent.mjs; app/planner.mjs and app/executor.mjs; contracts/MandateVault.sol. Hosting adapter: scripts/hosting/.

03 Agent Logic

From language to a bounded proposal

Groq is configured with openai/gpt-oss-120b. The adapter sends a short bill summary and accepts a strict JSON response describing an intent, known draft identifiers, a question or a correction. Local validation rejects unsupported shapes and unknown references. Invoice extraction proposes copied fields; applying a correction changes the draft and its review state.

The conversation layer distinguishes a status question, clarification, correction, unsupported request and payment request. It keeps the context of a clarification answer, while checking fresh ambiguous references. A new payment instruction can leave a read-only conversation context, but the resulting plan still passes through the execution-confirmation screen.

Deterministic planning

The planner obtains the relevant draft versions, confirmed approvals, vendor revisions, policy state, vault balance and spending for the UTC day. It sorts candidates by owner priority, then due date, then stable bill key. A due date determines ordering; an approved payment window determines when execution is allowed.

```
Read current draft and chain state
Sort by priority, due date, stable key
For each bill in that order
  Calculate eligibility using integer units
  Add eligible bills and update projections
  Preserve the checked reason for deferred bills
Return the plan for review
```

Amounts use integer base units throughout parsing, planning and transaction handling. No floating-point token arithmetic is delegated to the model. Reasons and next actions come from the checked application state, so the UI can explain a deferral without asking the model to decide whether a contract rule passes.

Queue and outcome handling

A confirmed plan becomes an ordered queue. Before each send, the executor refreshes state and simulates the fixed call. It records the transaction identity before broadcast, then verifies the resulting receipt. If processing becomes uncertain, the queue stops and recovery follows the original transaction reference. The security assessment covers the enforcement and rollback boundaries; Execution Evidence shows the observed results.

Reproduction parameters for the measured model run are retained with the evaluation freeze in the evidence report. Agent handling is implemented in app/agent.mjs, app/model.mjs, app/planner.mjs and app/executor.mjs.

04 Implementation and Setup

Pinned implementation environment

Area	Recorded versions
Runtime and contracts	Node.js 25.2.1; Solidity 0.8.37 targeting Cancun; Forge/Anvil 1.7.1; OpenZeppelin Contracts 5.6.1
App and documents	ethers 6.17.0; esbuild 0.28.2; csv-parse 7.0.3; PDF.js 6.4.299; pdf-lib 1.17.1
Browser checks	Playwright 1.64.1 with an isolated test browser
Hosted persistence	@neondatabase/serverless 1.2.0, pinned separately in scripts/hosting/package-lock.json

Start a fresh local demonstration

Extract the source deliverable into a new folder. Install the locked dependencies and build the contract artifacts before starting the local app:

```
npm ci
npm run build
npm run dev
```

The app opens at http://127.0.0.1:4173 and its disposable chain RPC at http://127.0.0.1:8545. Load the synthetic samples, review the source, approve the local bills and Resume. Preview without AI produces a labeled manual plan. The developer wallet/RPC accounts belong to this local chain; they are not public-network owner credentials.

The source README contains the complete check commands and platform notes. Tests write generated artifacts, so use a disposable extraction when checking reproducibility. Live-provider tests are separate commands and require the explicitly selected provider configuration; they are not needed for the local manual demonstration.

Hosted inspection route

Mandate hosted workspace

The reviewer route is documented once in competition/JUDGE_REVIEW.md. It explains private password delivery and how to inspect saved records. Hosting operations and runtime-variable setup are in competition/HOSTING.md; they are outside the local quick start.

Repository: https://github.com/arifshekhk8/chainsentinel-mandate (private). The source ZIP is the self-contained code review route; it does not require a repository invitation.

05 Verification Approach and Scope

How the implementation is checked

Contract tests call the vault directly, including attempts that bypass application planning. Application and agent checks exercise intake, review state, ordering, execution and recovery. Browser checks follow the user flow with disposable local accounts. Hosting checks separately examine access and persistence. This layered method helps identify which boundary each result actually supports.

The fixed-case model evaluation checks extraction, task outcome, explanation and interaction separately. Its saved inputs, model settings and grading remain available for inspection. Numeric tables, sample sizes and observed misses appear only in Execution Evidence, which also distinguishes local checks from public-network activity.

What the completed prototype establishes

The recorded workflow completes the intended approved-bill task and produces matching receipts. The local fixed-case regression met its numeric acceptance targets, with a safe completion miss retained. The public interaction also exposed an answer-completeness issue. These observations support a working prototype and identify further work; the evidence report provides the measurements and their qualifications.

Current scope and further work

This version handles supported CSV and text-PDF inputs for a single business. General OCR, recurring bills, multiple businesses and real-fund operation are outside scope. Next development priorities are broader invoice layouts, a human workflow study and the access/signing improvements identified in the security assessment.

Companion report navigation

Question	Primary report
Who may approve, execute or stop work?	Security Self-Assessment — Permission Model
How are hostile input, secrets and data handled?	Security Self-Assessment — Security Tests; Data and Privacy
What happened in the measured runs?	Execution Evidence — request trace, transaction register and evaluation
How can a judge check a result?	Execution Evidence — Reproduction Guide and log ZIP

Design and code references: ARCHITECTURE.md; REQUIREMENTS.md; TESTING.md; app/; contracts/; scripts/hosting/. The document describes the shipped prototype; test measurements and residual-risk conclusions remain in their primary reports.